

Active InferAnt Stream 009.1 ~ How to Accelerate? Active Inference Mountain

ActiveInferAntStream | How to Accelerate? Active Inference Mountain Car in RxInfer.jl | 1:30:10

All right, it's December 26, 2024.

This is Daniel Friedman.

It's Active Inference Stream 009.1.

How to Accelerate Active Inference Mountain Car in rxinfer.jl.

And hope everyone is having a good end of year season.

Looking forward to different updates, different live chats, see what happens along the way.

I'll start the stream off with making a GitHub push.

All right, while that's happening, I'm going to head over into Cursor.

This is what we're going to be exploring, covering today, which is modifying an rxinfer package example made by the developers.

And moving it from a notebook format into a script format, adding some more visualizations and analyses, and then doing a setup script, hopefully to make it easier to get it going on your computer, and meta-analysis.

And right now there's a big meta-analysis happening, but we'll look at some past runs that finished just a few seconds ago.

Okay, so check the GitHub.

This is based on the current version 3.8 of reactivebase slash rxinfer.jl.

And just to keep it separate and clear, made a fork or a branch from the exact version today.

And then put all of the work that I'm about to go into, into this examples folder slash mountain car.

So it's a branch on rxinfer.jl.

And in the video description are all the links.

Okay.

So GitHub push worked.

Now to start another standalone and meta-analysis.

So here's where I've gotten to after many speed bumps and fun along the way.

This is in cursor.

Right-click, open an integrated terminal.

First, if you haven't, run the script Julia setup.jl.

Probably also other settings will need to be bumped up against on different settings.

Also, I brought it up, synced it to the current version just after building part of it.

So there's probably a lot of inefficiencies.

It's, of course, very cursor slash Claude-esque.

So there could be a lot of redundancy, a lot of ways to go.

But I'm going to run the setup script.

Then we'll look at the standalone and the meta-analysis.

Okay.

So this is, as mentioned, branches off of the rxinfer example active inference mountain car.

Okay.

Let's just briefly look at setup.

See what's happening while it's finishing this.

There could probably be a lot of sorting of this.

Different packages are installed.

The rxinfer keystone package is installed from the local GitHub repo.

So probably it would take some other different way to export functions in a different way.

Calls, probably some set of automatic dependency installations.

Pre-existing, explicit calls that I typed in.

Like most recently, this stats plots.

So definitely figure out what else could be changed to get this working on your own computer.

But at the end, I hope it accurately says, yes, rxinfer's functional.

Some of the core packages that we need are loaded.

Here's the Julia version project path rxinfer path.

And then how to run the scripts.

And that can be probably developed.

Okay.

So that's the setup.

Then there's the standalone Julia.

Mountain car.

Standalone.

This one has today's date and a file name.

Just to give it a static version as a single file snapshot.

But the meta analysis is much more scattered across methods scripts.

So the single mountain car standalone has the dates.

Okay.

Okay.

So let's look before we go into the code.

How they set up and characterize the mountain car problem.

Okay.

So it's written in this sort of notebook, blog post, multimedia style.

Active inference mountain car.

This is from the rxinfer jail documentation.

A group of friends is going to a camping site that is located on the biggest mountain in the Netherlands.

They use an electric car for the trip.

When they are almost there, the car's battery is almost empty and is therefore limiting the engine force.

Unfortunately, they are in the middle of a valley and don't have enough power to reach the camping site.

Night is falling.

And they still need to reach the top of the mountain.

As rescuers, let us develop an active inference AI agent that can get them up the hill with limited engine power.

Okay.

Okay.

Okay.

Before we go into their technical details, here's just a single perplexity prompt.

Give an overview of the mountain car problem from control theory at four level slash types of depth.

Plain, archetypal, ultra technical, and esoteric.

And also generated some images.

So we can generate some more images now.

Or people can suggest other prompts.

So plain understanding.

The problem involves an underpowered car stuck in a valley between two hills.

The car's engine is too weak to climb directly up to the right hill to reach a goal flat.

Instead, it must swing back and forth, building up momentum by driving up the opposite hill first until it gains enough energy to reach the target.

Archetypal perspective.

This scenario exemplifies the fundamental principle that direct paths aren't always optimal.

The car must first move away from its goal, driving left to ultimately reach it on the right.

A counterintuitive strategy that demonstrates how complex systems often require indirect approaches.

The problem has become a classic test case for reinforcement learning algorithms because it requires learning this non-obvious solution.

So the active inference approach that RxInfer is going to implement doesn't have a trained reward or proposed reward function on some data set of successful or not successful traces.

It's going to be directly from the distributions.

Okay, technical analysis.

We'll see in more specifics.

Getting into the scripts.

And then I'll be the last one.

Esoteric implications.

The mountain car problem transcends its mechanical nature to represent a metaphor for optimization in complex landscapes.

The system demonstrates the necessity of building potential energy through oscillatory behavior.

The interplay between kinetic and potential energy transformations.

The emergence of sophisticated behavior from simple physical rules.

The relationship between local optima, valley, and global optima goal.

The problem's enduring relevance stems from its ability to challenge both traditional control methods and

modern machine learning approaches while maintaining mathematical elegance in its simplicity.

Okay.

So the goal for this stream, and then we'll look more at the specifics of the mountain car and the code itself.

To learn about RxInfer, hopefully, correctly, or at least helpfully implement one useful experiment.

To learn about Julia and all these different tools.

And shout out to Andrew for the early Julia adoption and insights.

Here is using Cursor 44.8.

RxInfer 3.8 or 2.

Julia 1.1, 1.2.

And then for the LLM in Cursor, Claude 3.5 Sonnet.

2024, 10.22.

All right.

Here's the finished single standalone that we ran, started during the stream.

It is going to implement two different simulations with the same physics and two different approaches to strategy.

First is going to be the naive approach, which always pushes to the right.

And then the second is going to be this active inference approach with a given look ahead period.

And this script goes into the outputs.

So here's the new one in green.

Here are the outputs of the single simulation.

And this will give an intuition for what the single standalone does.

First comparison, docif.

So here, the single parameter combination that is used was chosen.

It's, I think, similar or proportional or the same to what was used in the original example notebook.

And there's two different dots representing the cars position simulation on this landscape.

And we'll look at how physics is given on this landscape.

So the rules of physics are specified and the elevation and the potential and the kinetic energies are calculated in a sort of basic physics simulation style,

not simply modifying some attribute of the direct landscape function itself.

And they both start at negative 0.5 and they're both trying to get to positive 0.5.

The colors will change, I think, throughout.

But naive control is red and it drives right and it oscillates a little bit but reaches a fixed point where the friction and the gravity are equivalent to the engine's limiting force.

And that reaches this stable point.

The blue active inference agent has this anticipatory simulated trace.

And I think it's definitely worthy of another long investigation.

How does it distributionally fit this policy anticipation?

And what is the computational complexity as we scale the time horizon of inference?

But we can see that with this preference for being higher up, the active inference agent can see that it can get higher up to the left than it is on the right by moving that way.

And from already being up on the right slope and oscillate its way to the goal position.

And you can imagine that there's going to be combinations of physics and engine force and goal position that are easily or impossible to attain.

Going through the rest of the images in this single case output.

But first, I'm going to start the...

Well, let's just go through all these first.

Okay.

So this is the action distribution.

The naive control is always picking that one value to go right with one.

Whereas the active inference, now it's in tan or red.

It has...

At least, I don't know if I'm perfectly implementing what the notebook does.

But at least it does have some policy variance.

And one that for this parameter combination does get it up to the goal.

Action comparison.

So here's another way to look at that.

This is the naive control.

Now in blue.

With the y equals 1 policy function output.

Active inference agent having this more back and forth control policy swinging between 2 and negative 2.

I'm not sure if this is scaled to a different number.

But none of this is meant to say that this is the best fit.

It's really just a first step towards figuring out what kinds of tools and interfaces for generative models and for Rx and FUR

can we use to start doing some useful parameter sweeps across different regions of parameter space and structure models.

Okay, these are state action maps.

So here's position on the x-axis.

Velocity on the y-axis.

There's another one that makes sense.

Yeah.

So here's the trace of the naive noun blue and active inference showing these different trajectories.

They start out with zero velocity at negative 0.5.

Here's a version where the vectors are added.

The naive control follows this blue spiral to its fixed point attractor.

The active inference agent has this oscillatory behavior reflected with a loop, not just a spiral,

that ends up reaching a higher net position via increased velocity, which is what it needs to get up the hill.

Here's the predictive element of the active inference agent.

Here's velocity distribution.

There's a wider range of velocities ranging from the negative to the positive further in the active inference agent.

Energy comparison.

This is not like variational free energy.

This is the kinetic minus the potential or kinetic and the potential energy.

So we can look at this calculation though.

And energy efficiency, I think, is the integral or something of this, but not exactly sure.

Okay.

So this state action map is showing that trajectory and then mapping overlay with which action was taken.

So starting here, 2.0 was used.

That's moving to the right.

And then abruptly, the policy switches to negative 2.0.

Then policy switches here.

And then the position is approached.

And I think that's an interesting question.

Like, to what extent does it have inertia to meet the goal?

Or does it engage in a sort of Zeno's paradox slowing down as it converges if it has the engine control?

So it has to be kind of like a run past the finish line situation, at least in terms of the explicit preference distribution.

Otherwise, you get sort of slowing behavior leading up to the goal.

I don't know.

That could just be a total artifact of this hill.

Okay.

That's the single standalone output.

Okay.

So here's what it's going to look like here.

It was 200 simulations at 200 times.

Definitely interesting to profile.

How much is the startup time for one app model?

And then is there any other further caching?

Because starting the first one seems to take longer as the sort of Rx and firm machinery comes online, I guess.

But we'll look at the code biz too.

It should be interesting.

And again, it's all just initial takes for hopefully people and others to improve on it.

So here's the config.tala meta-analysis.

So there's two knobs in this parameter grid sweep.

The force limit of the engine and the friction coefficient of the landscape.

The gravity constant is kept constant.

For each of the engine force and the friction coefficient, you give a minimum and a maximum.

So I think the one that just ran, maybe it was this one.

I don't know.

Oh.

Let's see.

7.17.

We'll just run a clean one.

So minimum and maximum force and friction.

Let's do a higher resolution.

1 to 27.

0.01 to 27.

So this is going to do 15 times 15,
multiplied by the number of episodes per parameter combination.

So that can be used for technical replicates.

So then that parameter combination, we can understand the variance profile of it too.

I'll set that to 2.

Maximum steps per episode.

Let's leave it at 100.

Planning horizon.

Leave it at 40.

Initial position.

This is also position and velocity.

Third would be acceleration, etc., etc., etc.

Snap, crackle, pop, all that.

Div, grad, curl.

Target state is the position and velocity preferences.

So I think, again, interesting to see, are those convergent or does inertia carry some plotting?

Here, kind of as an experiment, kind of as a bit of a local coding optima,
some of the plots are in ASCII plain text.

And then some output.

So 15.15.

So that was just a sort of recap.

Run setup.jl.

Confirm that RxInfer works.

Then run

just the standalone.

Mountain car standalone 12.26.24.

And that will do the simulation.

We did that.

Close that window.

This is another

standalone.

Now, clearing.

Now we're going to run

the meta analysis.

So, hopefully this will work.

Meanwhile, let us explore the CodeViz

briefly.

Visualization.

So, not sure if this is accurate.

This is TBD with RxInfer experts.

But here, the prompt for this

graphical visualization was

I'll modify the prompt just so we have three free flows left for this out of five.

Here's the meta analysis.

Let's close that one.

Meta analysis plugging away on the terminal.

We'll see if it has to be

D or rebugged.

And then we'll do a graph is

free version on the free on this flow visualization.

Emphasize the component of active inference and the

free energy.

Okay.

Looks like we passed the first phase.

So, this loaded the configuration

toml.

Set up the

parameter batch

objects

plus methods.

So, now we'll see how the simulations go.

The first
part
I think takes longer
and then the second
it hits the second rate.
I'm not sure which
I think might
maybe does the
all the active inference ones
then all the
naive
and the
so
we'll see how long it takes to click to
end percent
and then see how long it takes to finish.
Okay.
Meanwhile
here was the graph this.
So, the
the prompt was
again
I hope
slash don't know
this is accurate
would be
really interested to explore
how we could develop
these methods
and get some
stable versioning
if these are accurate
and
use them for learning
Rx and Ver.
So,
trace how a probabilistic model specification
transforms

transforms from
at model macro definition
through node creation
to final compiled graph structure
including node contraction optimizations.

Emphasize the component of active inference
in the free energy principle.

Not sure it's not a matter.

So,
here is
ostensibly looking through
parts of this package.

Not sure how much
of a rocket
and all these other pieces
are in there
but, you know.

Here's some references
in the code.

Again,
I'm working in the
Rx and Ver branch.

So, here
it does find
a place
in the documents.

So,
the
app model macro
is passed
to
graph
PPL
package.

Here's a section.

This is kind of like using
cursor
slash

codeviz
as a rag
for code
and docs
facilitated
because the
Rx and Ver
documentation
is on
their
GitHub
as well.

So,
then
let's see.

App model
macro definition
uses graph
PPL
and that gets
customized by
reactive
message passing
graph
PPL
backend.

Okay.

Maybe.

Transformation
pipeline
compose
simple operators
folds
mathematical
expressions
and adds
brackets
to ensure

correct

argument

handling.

So,

the

app model

parameters

and

syntax

get kind

of unpacked.

Inject

tilde

right-hand-side

aliases

that

creates

nodes

and variables

based on

model

specification.

from the

nodes.

So,

this is the

statistical

base graph,

I guess.

Interesting.

Although some

nodes in

Rx and

Fer.jl

already come

with their

own

meta-structure,

users have
the flexibility
to define
different
meta-structures
for those
nodes and
also for
custom
ones.

A meta-structure
is created
by using the
struct statement
in Julia.

Okay.

That is
converted to
a factor
graph.

That's sort
of the

post-2017-ish
work with
Burt,
DeVries,
Friston,
Parr.

Here's the
free energy
optimization
component.

Okay.

So,
the factor
graph
construction,

I guess,
whether in the
scheduled
Forney
classic
non-reactive
style,
or
ostensibly,
but not
necessarily
in the
Rx and
Fer
reactive
message
passing
setting.
Probably the
message passing
can be
scheduled
or reactive
differently.
Here's
variational
free energy.
Variational
free energy
then defines
a functional
objective that
includes the
model and a
variational
distribution over
the latent
variables.

Not sure if
this is going
to always be
the case for
any graph or
how that
comes to be
for a given
factor graph,
but these are
just questions
to learn from
Explorer.

And then
here's a
specific
bet
free energy.

I know
Dave told
me it was
one way or
the other,
but probably
I said the
wrong one,
but the
bet the
approximation.

Okay,
meanwhile,
21% done
with this
225
times something
meta-analysis
sweep.

The BFE

can be
iteratively
optimized for
each individual
variational
distribution in
turn.

Optimization
of the BFE
is thus
more manageable
than direct
optimization
of the
VFE.

VFE.

So they
use also
these
methods
that in
the
RX

infer
group
this year
we kind
of
talked about
as the
what are
they called
the six
functions.

So they
are a
convention
which provides

interpretability
and structuring
for RX
and for
action
perception
loops
by using
a canonical
set of
methods.
and thanks
to many
of the
participants
throughout
this year
in getting
the documentation
doing all
these
experiments
too.

So I
don't know
how accurately
I've implemented
it but here's
what we
had and
there's
different ways
to do
it.

So in
the
structured
scheduled

single
time step
all every
node
one time
step
factor
graph
logistics
it's not
reactive
it's like
moving the
whole
simulation
ahead
one
click
the
I mean
what I
understand
of
with the
reactive
message
passing
situation
is that
it brings
some
computational
reactive
methods
to
the
local
variant

of
a
free
energy
the
free
energy
that
enable
variational
inference
to be
done
in a
flexible
and
effective
way.
Okay
node
contraction
oh
the sixth
function
so
once
into
the
reactive
setting
whereas
in the
scheduled
version
it was
just
iterate
through

this
list
of
operations
let's
do a
devrise
this is
graphical
brain
belief
propagation
and active
inference
first
in par
devrise
2017
figure
one
showing
how
one
time
step
updating
of
the
POMDP
graph
up here
translates
to
these
analytical
updates
occurring
in this

scheduled
order
on this
Forney
graph
which is
a
transformation
of
kind of
nodes
into
edges
and vice
versa
between
the
Bayes
and
the
Forney
graph
and
there's
also
recent
other
papers
and
live
streams
and
such
but
this
2017
is
just

laying
out
the
relationship
between
the
single
time
step
of
Bayesian
updating
and
the
update
rules
that
can
be
scheduled
on
that
Forney
graph
so
then
reactive
move
that
into
a
much
more
general
situation
where
different
processes

can
be
playing
out
with
different
logics
and
speeds
so
there
could
be
like
one
sensor
at
100
hertz
and
one
at
1
hertz
and
you're
neither
sampling
too
much
or
too
little
however
then
there's
this
question

of
okay
well
we're
going
to
specify
the
app
model
using
the
rx
and
for
app
model
syntax
looking
like
this
here's
transition
function
modeling
transition
due to
gravity
and
friction
so
updating
position
and
velocity
given
the
physics

the
structure
of
affordance
or
policy
in
action
space
um
inverse
engine
force
not sure
why it
needs to
be defined
here
this
is
i hope
just
putting
out
some
stabs
and
pheromone
trails
into
the
dark
for
people
to
clarify
perform
techno

invective
make it
clearer
make other
documentation
all these
kinds of
things
i'm
and i'm
not sure
if i've
completely
perfectly
mapped it
into the
script
form
but
from
this
example
this
is
the
proper
reference
version
the
at
model
function
for
the
agent
type
like
mountain

car
is
defined
like
in
this
generative
model
specification
language
where
for
time steps
during
the time
steps
of the
simulation
there's
a
direct
distributional
updating
occurring
so that
just
defines
all of
the
updates
to
occur
one
could
imagine
that
doing
the

order
differently
would
matter
so maybe
it could
be as
simple
as just
these
happening
but
the
order
could
matter
and you
might not
want
the
order
to
matter
in
exactly
the
same
order
as
the
code
are
lined
but
rather
to
take
this

all
at
once
representation
and
then
further
just
from
a
package
perspective
it's
desired
to
have
more
of
a
flexibility
with
how
you
call
the
statistical
distribution
so
that
again
those
different
logics
can
be
hooked
up
to

what
is
called
otherwise
if
it's
only
this
expressive
and
none
of
those
further
steps
were
taken
then
this
could
just
do
something
as
simple
as
iterate
through
in this
order
but
then
to
put
in
logics
well
like

actually
do
this
one
twice
as
fast
or
something
like
that
it
would
be
needing
the
kind
of
rewrite
that
would
result
in
some
of
these
steps
so
so
so
technically
the
agent
goes
to
three
phases
act

execute

observe

that's

one

step

infer

and

slide

kind

of like

act

infer

serve

act

execute

observe

in this

step the

agent performs

an action

onto the

environment

at time

t and

gets

another

gets

an

observation

in

exchange

these

observations

are basically

the prediction

of the

agent on

how the

environment
evolves
over the
next t
time
step
so
act
and get
an
updated
position
momentum
infer
after
receiving
observations
the
agent
starts
updating
its
internal
probabilistic
model
by doing
inference
it
finds
the
posterior
distributions
over
the
state
and
the
control

distributions

three

slide

after

updating

its

internal

belief

the

agent

moves

to

the

next

time

step

and

uses

the

inferred

action

from

the

previous

time

step

to

interact

with

the

environment

and

that

is

where you

pick

back

up

with
one
so
this
is
the
kind
of
oota
to
inactive
etc
and
these
steps
are
kind
of
like
mid
level
functions
explored
on this
page
here
with
six
where
there's
a
future
or
here

it
says

the
act
and
some
of
this
might
be
from
the
earlier
slightly
different
version
it's
just
all
sort
of
directionally
in
the
ballpark
that
because
the
graph
doesn't
have
this
intrinsic
scheduling
to
it
then
and
you
can

have
so
much
flexibility
with
a
custom
scheduling
custom
nodes
message
rules
all that
kind
of
stuff
that
you
need
to
bring
how
the
graph
is
called
and
updated
and
operated
more
explicitly
so
that's
a good
thing
it's
just

that
it's
still
slightly
evolving
with
many
of
the
package
semantics
and
examples
and
norms
and
there's
probably
many
patterns
to
explore
and
not
necessarily
some
sequence
or
repertoire
of
these
mid-level
possibly
semantic
or
sub-agentic
like
steps

that give
a lot
of nuance
with
describing
is it
OODA
or is
it
ADOO
or is
it
OOD
OOD
OOD
etc
okay
coming
back
to
see
other
simulation
okay
52%
done
on
the
meta
analysis
so
that
was
um
that
was
the
code

viz
let's
do
another
code
viz
help
us
totally
understand
how the
standalone
mountain
car
let's
see if
this works
help us
totally understand
how the
standalone
mountain
car
script
calls
every
minute
particular
and
etc
etc
leaves
thank you
55%
done
re-index
then
we'll

look
we'll
see
this
code
viz
then
we'll
look
at
the
output
of
the
meta
analysis
okay
but
briefly
just to
look over
what they
get to
so
they
describe
the
analytical
process
formalization
for the
mountain
so
again
it's
not
a
proposed

reward
or utility
function
or anything
trained on
data
it's
just a
distribution
that's
physics
grounded
using
just
simply
a
gravitational
force
on the
hill
landscape
depending
on the
position
and
that's
implemented
also
using
possibly
some
special
characters
and some
special
Julia syntax
that will
take some

learning and
some package
specific
syntax
so
to be
learned and
better understood
but here's
where they
lie out
the functions
that create
physics
and create
world
and the
sort of
world
generative
process
element
they have
a static
image
just to
confirm
that they've
initialized
everything
here's
the
naive
approach
this is
the
naive
I think

I have
implemented
it slightly
differently
with just
having it
all output
a one
each
time
step
maybe
they're
prefilling
it as
well
and then
here's the
visualization
of just
the
naive
policy
having
that
initial
ramp up
then
fixed
point
here's
the
active
inference
generative
model
okay
and

we'll
read
now let's
help them
solve the
problem with
an active
inference
approach
particularly
we create
an agent
that predicts
the future
car position
as well as
the best
possible
actions
in a
probabilistic
manner
so not
from
data
not from
data
sets
we start
by specifying
a probabilistic
model for
the agent
that describes
the agent's
internal beliefs
over the
external dynamics

over the
environment
the generative
model is
defined as
follows
so here
are the
equations
and in
the
github
for
basically
the
variables
in the
base
graph
and I
would look
forward to
using some
of these
new
visualization
methods
to
visualize
the
factors
and the
app
model
from the
notebook
visualize
the

standalone
and the
meta
analysis
to
confirm
that
the
model is
being
analyzed
correctly
okay
now let's
look at
standalone
well
let's just
finish
with
the
what they
have in
the
documentary
here's
the
variables
as
defined
here's
the
key
rx
infer
at
model
macro

at
model
macro
and
meta
blocks
are
used
to
define
the
probabilistic
model
and the
approximation
methods
for the
nonlinear
state
transition
functions
respectively
in addition
the
beliefs
over the
future
states
up to
t
steps
ahead
of
the
agent
is
included
so

this
is
like
the
agent
class
equivalent
which
we
looked at
briefly
before
defined
as
those
distributions
over
those
variables
after
specifying
the
generative
model
let's
create
an
active
inference
agent
for
the
car
technically
the
agent
goes
through

three
phases
so
here
again
are
those
mid-level
semantic
bundles
of
sort
of
distributional
updating
cognitive
subagentic
process
etc
that
are
being
projected
into
this
three
cycle
act
execute
observe
infer
slide
but
that's
again
as far
as I
understood

from
COBUS
and others
and
learning
this
year
that's
just
a
convention
and
other
orders
or other
kinds
of
that
could
be
a
good
place
to
play
with
this
simulation
and
again
to
confirm
through
the
flow
of
the
simulation

with
the
code
viz
how
the
notebook
the
standalone
and
the
meta
analysis
might
be
correctly
or
incorrectly
!
applied
so
in the
cell
below
we
create
the
agent
through
the
create
agent
function
which
includes
compute
!
act

slide
and
future
so
here's
future
but it
wasn't
mentioned
above
here
is
where
the
reactive
message
passing
component
is
called
not
exactly
sure
what
they
mean
about
the
message
so
again
just
a
learning
artifact
to
try
to

understand
the
extremely
interesting
potential
and
directions
we can
go
with
the
kind
of
flexibility
that we
have
with
Rx
and
fur
and
how
cool
their
direction
and vision
for the
open
source
element
is
too
so
here's
the
create
agents
method

now
it's
time
to
see
remember
now
it's
time
to
see
if
we
can
help
our
friends
arrive
at the
camping
site
by
midnight
question
mark
here
is
the
Julia
syntax
there's a
combination
of
initialization
methods
parameterization
agent
creation

further
parameter
specification
step
through
experimental
protocol
then
this
discrete
time
loop
for the
number
of
time
steps
as the
octave
inference
simulation
is going
to go
and then
here's a
very concise
animation
method
and so
here
it's a
little bit
different
but it's
the same
kind of
general
picture

that
is shown
here
different
scaling
colors
but
that's
what is
happening
in
outputs

plays out
with this
example
okay
all right
let's look at
the code
viz
for the
whatever we
prompt
okay
83%
done
with
the
big
meta
analysis
okay
so here
we go
here's
mountain car
standalone

1226

2024

first

loads

config

dot

tunnel

like

I don't

even know

if it

does

it

does

that's

one

it

also

configures

a shared

logging

function

and a

visualization

probably

data

collection

arraying

function

so then

these

config

informed

visualization

and logging

methods

are

holding

the traces
of these
two agents
as they're
simulated
within a
shared
parameter
configuration
like physics
and friction
that's each
cell of this
meta analysis
it's ongoing
and each
of these
agent
folders
we could
say
has like
a
naive
control
agent
for that
physics
combination
and
active
inference
agent
for that
physics
configuration
the
parameterized

sort of
setup
for
the
simulation
and
everything
loaded in
place
which can
often be
a lot
of the
hardest
part
sometimes
cursor
plus
LLMs
knocks it
out of
the park
other
times
it's
just
two
steps
backwards
one
step
forward
mountain
car
is
called
interesting
mountain

car
dot
jail
even
here
looks like
it goes
to code
of conduct
here's
the
readme
let's
look at
that
on
github
let's
look at
some of
these
methods
wow
though
interesting
okay
95%
done
with
meta
analysis
here's
here's
the
mountain
car
model
method

so here's
standalone
standalone's
parameters
are
I don't
know
if
terminally
I'm sure
it could be
restructured
are
written
at
300
lines
in
just
wanted
to
start
there
it's
a
highlight
tab
so
here's
how I
understand
what's
happening
in the
standalone
it's
defining
this

as
the
mountain
car
module
and
and
I
think
a
better
package
reference
and
function
reference
could
go
a
long
way
here
at
the
package
and
at
the
specific
method
level
rx
infer
and
reactive
message
passing
sub

kind of
second level
consultants
are called
and
some of
the other
packages
that are
needed
visualization.jl
is brought
in for
some methods
also
this
file
exports
public
functions
so
other
scripts
can
call
include
on
this
script
and
get
these
functions
here's
some
all right
here we
go

completed
900
simulations
with 80
successes
okay good
so that
means that
some of
the
simulations
should have
worked
some of
them
didn't
work
so at
least we
picked
some
interesting
area
not sure
if it's
going to
visualize
everything
properly
okay
here's the
app model
function
with like
the same
I hope
distributions
and such

here's the
create physics
here's the
create world
here's the
create agent
here's
calculating the
kinetic
potential
and total
energy
so here's
the
potential
energy
with
9.81
gravitational
constant
okay
no errors
from the
meta-analysis
but we'll see
what it ends
up outputting
what was
output
so here's
the physics
based
energy
calculation
not to be
confused
only to be
distinguished

from the
fact that
the
message
passing
updates
are doing
a free
energy
minimization
but this
is referring
to the
physics
I don't
know what
it's
if there's
a
just a
physics
phase space
for
the
Newtonian
mechanics
here's the
method to
call for
the animations
that's
the
module
then here
is when
that
standalone
is called

by itself
it calls
in all
these other
packages
here we're
at line
300
here are
the
parameterizations
these may
override
or
vice
versa
and
set the
shared
parameters
for the
naive
and the
active
inference
approach
here's
the
naive
approach
here's
the execution
of the
naive
approach
this is
sort of
a procedural

way
they
defined
just
again
like
these are
this is
like
an example
of
something
that
might be
totally
irrelevant
in a
given
moment
or
it
might
make a
huge
difference
in a
simulation
so here
they put
100
ones
or
n
naive
time
steps
of
ones

and
then
just
accepted
that
vector
here
possibly
more
error
prone
possibly
introducing
disastrous
logics
there's
a
procedural
generation
of
setting
the
action
at that
time
step
to
one
and
that
enables
some
of the
single
standalone
logging
here's
the active

inference
approach
so
it's
pre
figured
with
the
AI
namespace
subscript
here
is
the
compute
act
slide
future
agents
not
sure if
this
0.04
gets
overridden
we'll
see
maybe
in the
meta
analysis
here's
the
active
inference
simulation
in a
let

block
which
helps
with
the
I
don't
mean
impenetrability
security
but in
the
error
tolerance
and
logging
for
allowances
in
programs
so
different
sort of
footholds
entry points
for logging
and
understanding
and
for a
given
situation
maybe
more
or
less
there's
a lot

I
don't
even
know
so many
different
ways
to write
different
codes
so
here's
the
observation
of
the
position
and the
velocity
from
the
current
state's
modalities
factors
here
that's
the
initial
position
here
is
the
time
step
discrete
time
sequence

so
this is
kind
of
like
there's
the
topos
of
the
topology
of the
action
perception
partition
and then
this is
like
the
kairos
possibly
also
chronos
for
specifying
how
those
different
topological
aspects
of the
action
perception
partition
get
enacted
sequentially
in this

case
but
with
expressivity
to have
other
logics
underneath
these
just
current
transient
norm
specifications
so
set the
action
to the
act
function
push it
onto the
action
trace
and execute
the
action
then
observe
push the
observations
store
the
predictions
and
and the
future
call

check
against
the
target
update
beliefs
so maybe
that's the
sort of
running
through the
finish line
component
like it
checks
whether it's
actually at
the goal
event it
updates its
beliefs
but I'm not
sure
and logs
every 20
time steps
here's the
calculation
of the
energies
the
visualization
on the
analysis
of the
naive
and the
AI

comparison
plots
outputs
some
summaries
okay
so that
again goes
to the
outputs
folder
now let's
look at
so 17
okay
so here's
an ANOVA
but here's
some earlier
ones that
have many
more
let's try to
get these
back
first just
going to
explicitly
bring in
the
meta
analysis
and the
mountain
car
methods
okay
please

ensure that
all
ASCII art
and
non
ASCII
parentheses
quote
normal
plots
are
fully
output
by the
meta
analysis
then we'll
update the
toml
just to
do a
tiny one
just to
verify that
so we'll
do two
by two
so this is
kind of a
common
sort of
micro dev
loop
situation
with
of course
just
breaking

unworking
things
but
visualizations
sort of
trickling
off
needing
to be
fixed
and then
the sort
of
trade-off
between
pursuing
a local
branch
and then
pursuing
a deeper
branch
so for
example
here was
using the
ASCII
text art
so this
was on
450
for each
sweeping
so let's
see these
were the
plain text
we'll see

if we can
recover these
but you
know
hilarious
if we
can
hilarious
if we
can't
okay
here's
just a
sort of
summary
on the
overall
success
versus
failure
of naive
versus
active
inference
so
TBD
if it's
exactly
perfectly
doing what
the standalone
does
and again
if that's
exactly
the right
or only
way to

do it
in the
example
but at
least in
this
parameter
zone
active
inference
is doing
better
so there's
at least
some
cases
where
there's
a
better
policy
than
just
going
to
the
right
success
rates
so
here's
sort of
an
ASCII
art
visualization
not sure
if the

dimensionality

is the

right way

but it's

sort of

some

example

here's

the

difference

here's

some

different

plots

this is

why

it was

a local

optimal

because

it looks

kind of

sick

so I

thought

okay

it could

definitely

be

also

improved

but it's

also

like

it's

cool

it can

stay

parameter
analysis
so here's
success rate
versus
force
in the
naive
success
rate
not sure
why this
wouldn't
be a
monotonic
distribution
different
plots
like
plotting
this
by that
plotting
this
by that
okay
meanwhile
I'm going
to accept
blindly
the edits
that it
made
to the
meta
analysis
double
check

the
toml
we're
doing
a
much
smaller
situation
clear
run
again
back
to
the
earlier
outputs
heat
maps
is
empty
so that
was
one
of
the
non
ASCII
art
energy
analysis
so this
again
suggestive
that
the
simulation
really
is

happening
which
is
kind
of
the
question
of
these
different
cases
but
it's
limited
ability
with
ASCII
so here's
another
visualization
that's
it's like
the
control
effort
distribution
by
agent
type
almost
always
is at
100
but
then
there's
a few
other

ones
so
not
sure
if
those
are
spurious
or
how
those
come
to be
in
the
naive
control
effort
case
but
here's
this
different
distribution
okay
and
analysis
is
empty
so
here's
an
example
of
the
sort
of
extended

startup
time
of
rx
infer
okay
so this
will probably
have to be
resolved
but here
is setting
up
the
smaller
simulation
so it
sets
up
the
whole
batch
whether
it
has
to
do
two
or
two
hundred
thousand
and it's
going to
go a lot
faster
it's already
at 12%

okay
okay
so I'll
just sort
of read
some of
these
second
closing
parts
while we
finish up
this
meta
analysis
if you're
watching
live
though
write
any
comments
or
questions
in the
chat
okay
so here's
some
open
source
to
do
at
the
package
scale
just

verify
and
improve
you
know
anyone
who
wants
to
try
can
clone
the
repo
get
the
setup
dot
jl
working
on
another
setup
on
the
statistical
front
we
can do
better
statistical
description
work
visualization
and
analysis
within
and across

simulations
so
sort
of
distinguishing
and building
some
statistical
power
apparatus
for
technical
replicates
and
biological
replicates
or
experimental
conditions
19%
on the
simulation
on the
formal
and the
analytical
structural
side
well
one
approach
is from
the
initial
RX
and
fur
teams

notebook
and from
the
standalone
and from
the
more
meta
analysis
style
apply
generalized
notation
notation
GNN
style
which we
can actually
try this
in
this
stream
briefly
so we'll
leave that
one up
category
theory
here
added a
couple of
links
before
still
analyzing
the
meta
analysis

at least
that's a
good
sign
so
wanted to
highlight
two
recent
blog
posts
from
August
and
November
of this
year
by
colleagues
at the
Topos
Institute
so
the
earlier
one
is a
blog
post
by
Sophie
Libkind
and
David
Spivak
a
polynomial
account

of
Bayesian
update
so
here
they
describe
this
poly
tree
notation
this
is a
very
math
arty
and
useful
overview
for the
poly
category
theoretic
notation
and some
operations
and probably
a ton
more
for
Bayesian
update
and
later
they
did
work
including

more
of
the
action
side
and
then
here's
a post
from
November
of this
year
by
David
Spivak
describing
a paper
dynamic
task
delegation
for
hierarchical
agents
meta
analysis
finished
with
an
error
no
look
just
fix
it
thoughtfully
so
neural

wiring
diagrams
for
message
passing
in
multi
scale
organizations
sounds
to
connect
on a
lot
of
act
inf
topics
so
they
wrote
in
our
recent
paper
dynamic
task
delegation
for
hierarchical
agents
Sophie
Lipkin
and I
described
a
task
delegation

from
an
agent
to
a
team
of
subordinates
as
morphisms
in a
certain
operad
called
org
m
however
however
such
morphisms
include
a
great
deal
of
data
and
the
examples
we
gave
there
barely
scratched
the
surface
of
what's

possible
in
this
post
I'll
show
how
recumbent
neural
wiring
diagrams
with
arbitrary
feedback
and
axon
splitting
give
rise
to
a
message
passing
language
for
describing
certain
such
task
delegations
that
is
I'll
define
a
cyclic
operad
rnwd

and a
function
rnwd
arrow
!
right
now
what
are
you
doing
rnwd
and
there
could
be a
lot
of
connections
here
with
the
message
passing
I
want
to
just
highlight
those
see
if
anyone
is
interested
to
explore
that

direction
just
in
terms
of
making
okay
now
back
to
the
code
here
accept
the
changes
resync
it
clear
start
up
the
meta
analysis
again
on
the
sort
of
action
perception
behavioral
science
component
fitting
data
playing
with

different
noise
patterns
and
levels
and
then
also
figuring
out
some
other
strategies
that
can
get
plugged
in
other
than
the
naive
one
so
just
for
reference
this
is
the
I
believe
formerly
known
as
open
AI
gym

now
called
gymnasium
dot
for
rama
dot
org
so
mountain
car
here's
how
they
describe
this
the
environment
is
part
of
the
classic
control
environments
which
contains
general
information
about
the
environment
so
description
this
is
how
they

describe
it
on
that
gymnasium
situation
the
mountain
car
MDP
is
a
deterministic
MDP
markov
decision
process
that
consists
of a
car
placed
stochastically
at the
bottom
of a
sinusoidal
valley
where
the
only
possible
actions
being
the
accelerations
that
can

be
applied
to
the
car
in
either
direction
the
goal
of
the
MDP
is
to
strategically
accelerate
the
car
to reach
the
goal
state
on
top
of
the
right
hill
there
are
two
versions
of
a
mountain
car
domain

and
gymnasium
one
with
discrete
actions
and one
with
continuous
this
version
is
the
one
with
discrete
actions
first
appeared
in
Andrew
Moore's
PhD
thesis
1990
so
here's
just
how
they
do
it
so
that
could
be
cool
to

bring
the
RX
and
FUR
active
inference
agent
to
the
gymnasium
and
vice
versa
another
angle
here
look
into
the
links
go
to
the
multi
agent
page
in
active
block
inference
project
document
and
probably
other
and
better

sources
for this
too
but
this
is
just
from
years
ago
in
the
cyber
physical
case
and
the
ant
case
some
of
the
different
ways
that
hopefully
RX
environments
and or
similar
will
help
enable
with
the
expressivity
to
talk

about

a

sort

of

pure

blockchain

stegmargy

niche

modification

neighborhood

where you

have a

kind of

centralized

at that

scale of

analysis

hub and

spoke

to and

from

like for

example

in the

blockchain

case

people might

talk about

sending from

wallet A

to wallet

B

but

from a

message passing

perspective

it was

wallet A

to
the
blockchain
and
then
the
blockchain
update
changes
the
efficacy
at a
future
state
for
B
whereas
the
in this
case
maybe
explicit
is not
the best
term
here
but
spatial
collisions
and direct
interactions
are possible
as well
as
environment
interactions
so
okay

we're
back
to
the
meta
analysis
okay
already
it looks
like more
folders
have come
out
16
simulations
with 4
successes
so again
some of
these
should
okay
so
here we
are
within
this
most
recent
we'll
wait for
this
to
finish
and
then
we'll
look

into
this
most
recent
meta
analysis
okay
the
heat map
okay
okay
chaos
surprise
chaos

mouse
over
or
just
copy
and
paste
it
into
the
composer
please
thoughtfully
address
all
so
that
the
visualizations
of all
kinds
are
safely

plotted
and
fully
logged
output
okay
meanwhile
what is
happening
in these
folders
okay
heat maps
went
empty
but
here's
what came
out
before
that
okay
so
here
was
the
smaller
parameterization
with
these
other
ASCII
art
so
for
the
for
the

cyberpunk
solarpunk
RX
and
fur
enthusiast
this
one
oh
is
much
bigger
the
the
size
of
the
ASCII
art
can be
specified
in the
tunnel
plot
width
and
plot
height
okay
meanwhile
the
agent
is
chugging
away
over
here
okay

then
parameter
analysis
with
these
bigger
ones
so
!
not
exactly
sure
how
accurate
everything
but
okay
heat
maps
empty
that's
kind
of
part
of
the
issue
okay
interesting
with a
naive
energy
distribution
ASCII
plots
nothing
nothing
okay

accepting
these
visualization
updates
clear
let's
just
delete
the
ones
from
after
18
call
the
meta
analysis
it's
going to
run
it
so
how
can we
use
RX
environments
or
and or
others
to
model
this
kind
of
direct
communication
amongst

agents

low

road

or

high

road

to

model

transient

communication

like

propagative

radio

EMF

light

schooling

flash

blocking

or

sort

of

proximate

antenna

antennal

interactions

as well

as to

model

communication

through

spacetime

cones

thinking

more

about

topics

like

niche

modification
stigma
G
across
trial
learning
and
pheromone
psychophysics
in the
active
inference
case
COBUS
live in
the
dock
okay
so here
appears to
be
a key
part
of the
issue
so we'll
see how
much
this
interferes
with it
on this
new run
here
happening
here
but
there's

some
overlap
between
the
ASCII
art
let's
just
preempt
ensure
that
we are
not
quote
getting it
twisted
with the
plots
and
ASCII
art
plots
math
art
plots
please
check
over
everything
and
fix
and
trench
to be
100%
functionality
okay
checking out

the
readme
structure
of the
repo
standalone
implementation
physics
engine
octave
inference
agent
naive
agent
visualization
meta
analysis
framework
at least
the hope
of one
difference
measures
setup
with
setup
script
run
standalone
or the
meta
analysis
update
config.toml
hopefully
project.toml
the one
with all

the package
stuff
should be
taken care
of by
setup
but
submit a
request
or let's
figure out
how to
make it
simpler
and
here are
some of
those
key
packages
including
Rx
and
Fer
okay
all right
finish the
16
simulations
now
it's
plotting
so we
anything
with
18
see
but we're

already

kind of

we're

already

on another

okay

rerunning

it

starting

now

to

fix

but

basically

this is

the whole

coefficient

slash

question

how fast

how good

how

augmented

how clear

how well

documented

how aligned

how useful

how interesting

how novel

how valuable

so let's

let that

simulation

go

so let's

let that

simulation

go
okay
while the
meta analysis
is happening
let's
have
just a
more
folder
let's
just see
how it
does
with
who's
on first
in
who's
on first
write
a
please
write
a
comprehensive
absolutely
informed
and far
incited
flash
out
cited
parenthesis
indoor
outdoor
specialist
status

who's
on first
style
dialogue
or more
about
1920s
americana
baseball
usa
os
pacific
lore
and
all the
contents
of
the
feed me
and
mountain
car
active
inference
reactive
message
passing
scheme
as
we've
tried
to
implement
and
dot
dot
dot
dot

I'll
help
write
a
humorous
abbott
and
costello
who's
on first
style
dialogue
that
cleverly
weaves
together
classic
baseball
references
with the
mountain
car
active
inference
implementation

Let me write this in the who's on first.md file.

69% done with the meta-analysis.

Okay.

Okay.

Let's see if...

Okay.

We'll just do the one shot.

Okay.

Pushed it to GitHub.

Accept the changes.

Clear.

Run meta-analysis.

Reload the repo.

More.

More.

All right.

Just one shot, just so that we can close out.

Who's on first?

Mountain car edition.

Say, Costello, I've been meaning to tell you about this fascinating new project we're working on at the ballpark.

Oh, yeah?

What's that?

It's called mountain car active inference.

A mountain car?

At a baseball park?

What are you?

Nuts?

No, no.

It's a simulation.

We've got two agents trying to get up the hill.

Two agents?

Like baseball agents?

No.

One's active inference and one's naive.

Well, that's naive of them both.

Who's controlling them?

Exactly.

Who's controlling the active agent?

What's controlling the naive agent?

And I don't know is measuring the performance metrics.

Wait a minute.

Who's controlling the active agent?

Yes, precisely.

I mean the fella's name.

Who?

The agent controlling the car.

Who's controlling the active inference agent?

That's what I'm asking you.

And I'm telling you who's controlling the active inference agent.

Look.

Let me start over.

When you run the simulation, who plots the result?

Oh, no.

Who doesn't plot anything?

What plots the results?

I'm not asking what plots the results.

What does plot the results?

Creates beautiful heat maps, too.

I don't care about heat maps.

I want to know who's plotting.

No, no.

Who's controlling?

Not plotting.

Then who is plotting?

No, like I said, what plots everything?

All right, wise guy.

Let me try another way.

The car needs to get up the hill, right?

Right.

So who calculates the force needed?

No.

Who only handles active inference?

I don't know calculates the forces.

You don't know who calculates the forces?

No.

No.

I don't know calculates the forces.

That's what I'm asking you.

And that's what I'm telling you.

I don't know calculates the forces.

Then who does?

No.

Who does active inference?

I don't know does forces.

What does plotting?

And tomorrow handles the parameter sweep.

Parameter sweep?

You bringing in a groundskeeper now?

No, no.

Tomorrow is our batch simulation handler.

Today is not involved at all.

How can you run simulations if today is not involved?

Why would you want today involved?

I'm not asking why.

Oh, why is busy analyzing the energy consumption?

I don't even know what we're talking about anymore.

That's the spirit.

Now you're getting it.

I don't know.

It's a crucial part of our implementation.

Ah.

Ah.

Hey, wait.

I haven't told you about because's statistical significance testing yet.

Later that day.

Okay.

Let me just ask one simple thing.

Sure.

In the config file, who sets the initial position?

Oh, no.

Naturally sets the initial position.

Naturally?

Yes, naturally.

I should have stayed in the minor leagues.

Don't forget practice tomorrow.

We're running a full parameter sweep.

Distance screaming.

Only to return.

Tomorrow airs.

Let's see how far we got.

Okay.

Let's see how far we got.

Let's see how far we got.

Still no heat maps.

Okay.

I'll give one more minute or so if anybody wants to write a question in the live chat.

Otherwise, look forward to seeing what people think.
see where it's incorrectly or inefficiently
applying active inference with RxInfer
because this is a really promising direction.
There's a lot of interfaces for specifying simulations,
making versions of a textbook,
examples, all kinds of fun research and education
and application that RxInfer is enabling.
So I will accept this last edit,
make the last push.
Yeah, Composer, being able to edit across multiple files.
See, this is good.
It looks good,
but it's really hard to see just from a quick scan.
So I'll just leave it at that.
Close cursor.
Close cursor.

Make last git push.
Okay.
Verify that closing edits were sent.
All right.
Thank you, everyone.
Enjoy.
I wish you a good closure of the year,
beginning of the next,
good accelerations
in this continuous discrete complex scenario we're in.
Okay.
Thank you.
Bye.
Bye.